C I N E S C A N

Cinema Booking & Ticketing System

Complete Software Documentation

System Architecture · Database Design · Flowcharts · Developer Guide

Document Version: 1.0
Generated: June 2026
Classification: Internal / Developer Reference

1. Executive Summary

Field	Details
System Name	CineScan – Cinema Booking and Ticketing System
System Purpose	A full-featured point-of-sale and seat-reservation platform for small-to-medium cinema operators.
Problem Solved	Eliminates manual paper-based seat tracking, removes double-booking risks, generates barcode tickets, and provides management reporting.
Intended Users	Cinema Administrators (full access) and Staff / Cashiers (booking & scanning).
Core Technology	Desktop application (PHP/Web stack likely), MariaDB 10.4.32 database, phpMyAdmin for DB management.
Key Features	User authentication with roles, movie catalog management, multi-hall scheduling, interactive seat map, barcode ticket generation, booking lifecycle (Active/Used/Cancelled), walk-in and named customer bookings.
Database	Schema: cinescan | Tables: users, movies, cinema_halls, showtimes, seats, bookings | Server: 127.0.0.1 (localhost)

2. Complete System Overview

2.1 System Purpose & Scope

CineScan is a cinema management and point-of-sale application designed to handle the complete booking lifecycle from movie scheduling to ticket scanning. It manages three cinema halls, each with 30 seats (A1–C10, 3 rows × 10 columns). Administrators configure movies, halls, and showtimes; staff sell tickets and scan barcodes at entry.

2.2 Main Workflows

Workflow	Description
1. Authentication	User enters username + password → system validates against users table → role-based dashboard displayed.
2. Movie Management	Admin adds/edits/removes movies with genre, rating, duration, price, poster image, and hall assignment.
3. Showtime Scheduling	Admin creates showtimes (Start_Time, End_Time, Hall_ID, Movie_ID) with a default status of 'Scheduled'.
4. Seat Selection	Staff selects a showtime → interactive 3×10 seat map loads per hall → available vs. reserved seats highlighted.
5. Booking & Payment	Staff confirms seat + customer details → system generates unique barcode (CS + 12 hex chars) → booking saved with status 'Active'.
6. Ticket Scanning	At entry, barcode is scanned → system queries bookings table → status changed from 'Active' to 'Used'.
7. Reporting	Admin views booking history, revenue totals, occupancy rates filtered by movie, hall, or date range.

3. Software Architecture Analysis

3.1 Architectural Pattern

CineScan follows a layered N-Tier / MVC-influenced architecture with the following separation of concerns:

Layer	Components	Responsibilities
Presentation Layer	Login Form, Dashboard, Movie List, Seat Map, Booking Form, Scan Screen, Reports, User Management	User interaction, input collection, displaying data fetched from business layer.
Business Logic Layer	AuthController, BookingService, MovieController, ShowtimeController, SeatManager, ReportService, BarcodeGenerator	Validates inputs, enforces business rules (e.g. no double-booking), orchestrates data access.
Data Access Layer	UserRepository, MovieRepository, BookingRepository, ShowtimeRepository, SeatRepository	Raw SQL/PDO queries, CRUD operations, database connection management.
Database Layer	MariaDB: users, movies, cinema_halls, showtimes, seats, bookings	Persistent storage, referential integrity via foreign keys, AUTO_INCREMENT PKs.

3.2 Presentation Layer – UI Screens

Screen	Description
Login Screen	Entry point. Collects Username + Password. Validates locally then delegates to AuthController.
Admin Dashboard	Overview panel showing total movies, bookings, revenue, active showtimes, and quick-access navigation.
Movie Management	CRUD list of movies. Form fields: Title, Genre, Duration, Rating, Price, Poster, Hall, Status.
Hall Management	Lists cinema_halls (Cinema 1–3). Toggle Is_Active, edit Total_Seats, add new halls.
Showtime Scheduler	Calendar/list view. Select movie + hall + start/end time. Auto-status: Scheduled → Ended.
Seat Map	Interactive 3-row × 10-col grid per hall. Green = Available, Red = Reserved/Occupied, Gray = Sold.
Booking Form	Triggered after seat selection. Inputs: Customer_Name, showtime confirmation. Shows price and generates barcode on save.
Ticket / Receipt	Printable receipt showing Barcode, Movie, Showtime, Seat, Hall, Price, Customer Name.
Barcode Scanner	Input field (keyboard/USB scanner) for barcode. Validates and marks booking Used.
Reports Screen	Filterable table of bookings. Revenue summation per movie / date. Export option.
User Management	Admin-only. CRUD for users table. Assign Role (Admin/Staff), toggle Is_Active.

4. Project Structure Breakdown

4.1 Directory Tree

```
CineScan/
├── index.php                  ← Application entry point / router
├── config/
│   └── database.php           ← DB connection (host, user, pass, dbname)
├── controllers/
│   ├── AuthController.php     ← Login / logout / session
│   ├── MovieController.php    ← Movie CRUD
│   ├── HallController.php     ← Hall CRUD
│   ├── ShowtimeController.php ← Showtime CRUD + status update
│   ├── SeatController.php     ← Seat map retrieval + reservation
│   ├── BookingController.php  ← Booking creation, barcode gen, scan
│   ├── ReportController.php   ← Revenue and occupancy queries
│   └── UserController.php     ← User CRUD (admin only)
├── models/
│   ├── User.php               ← User entity + UserRepository
│   ├── Movie.php              ← Movie entity + MovieRepository
│   ├── CinemaHall.php         ← Hall entity + HallRepository
│   ├── Showtime.php           ← Showtime entity + ShowtimeRepository
│   ├── Seat.php               ← Seat entity + SeatRepository
│   └── Booking.php            ← Booking entity + BookingRepository
├── services/
│   ├── BarcodeService.php     ← Unique barcode generation (CS + hex)
│   ├── AuthService.php        ← Password check, session management
│   └── ReportService.php      ← Aggregation, revenue math
├── views/
│   ├── login.php              ← Login form UI
│   ├── dashboard.php          ← Admin/Staff dashboard
│   ├── movies/
│   │   ├── index.php          ← Movie list
│   │   ├── create.php         ← Add movie form
│   │   └── edit.php           ← Edit movie form
│   ├── halls/
│   │   └── index.php          ← Hall management
│   ├── showtimes/
│   │   └── index.php          ← Showtime list + add form
│   ├── seats/
│   │   └── map.php            ← Interactive seat map
│   ├── bookings/
│   │   ├── create.php         ← Booking confirmation form
│   │   ├── ticket.php         ← Printable ticket/receipt
│   │   └── scan.php           ← Barcode scan screen
│   ├── reports/
│   │   └── index.php          ← Revenue + booking reports
│   └── users/
│       └── index.php          ← User management (admin)
├── assets/
│   ├── css/style.css          ← Global styles
│   ├── js/seatmap.js          ← Seat selection JavaScript
│   ├── js/scanner.js          ← Barcode input handler
```

```
│   └── posters/              ← Movie poster images
└── database/
    └── cinescan.sql          ← Full schema + seed data
```

4.2 File Responsibilities

File	Purpose	Depends On	Called By
config/database.php	Defines DB constants (HOST, USER, PASS, DB_NAME). Returns a PDO or mysqli connection object used by all repositories.	All model files	None
controllers/ AuthController.php	Handles POST /login: reads $_POST, calls AuthService::authenticate(), sets $_SESSION['user_id'] and ['role'], redirects.	AuthService, User model	login.php view
controllers/ MovieController.php	CRUD: index() lists all movies; create/store add a movie; edit/update modify; destroy soft-deletes by setting Status='Ended'.	Movie model	movies/ views
controllers/ HallController.php	Lists halls, toggles Is_Active, updates Total_Seats. Validates seat count before update.	CinemaHall model	halls/ views
controllers/ ShowtimeController.php	Creates showtimes (validates no hall overlap), lists by movie, auto-updates status to 'Ended' past End_Time.	Showtime, Movie, Hall models	showtimes/ views
controllers/ SeatController.php	getMap($showtime_id): queries seats table by Hall_ID, overlays bookings to mark Reserved/Occupied. Returns JSON for seat map JS.	Seat, Booking models	seatmap.js, map.php
controllers/ BookingController.php	Core workflow: validate seat availability → call BarcodeService → INSERT booking → UPDATE seat status. Also handles scan/mark-used.	Booking, Seat, BarcodeService	bookings/ views
controllers/ ReportController.php	Aggregates bookings by date/movie/hall. Sums Price, counts statuses. Returns to report view.	Booking, Movie, Hall models	reports/index.php
controllers/ UserController.php	Admin-only CRUD on users table. Hash password on create/update (MD5 or plain in current impl).	User model	users/index.php
models/Booking.php	Booking entity class + BookingRepository: findByBarcode(), findByShowtime(), insert(), updateStatus(). Contains SQL.	config/ database.php	BookingController
models/Movie.php	Movie entity + MovieRepository: findAll(), findById(), insert(), update(), updateStatus().	config/ database.php	MovieController
models/Seat.php	Seat entity + SeatRepository: findByHall(), reserve($seatId), release($seatId), isOccupied().	config/ database.php	SeatController, BookingController
models/Showtime.php	Showtime entity + ShowtimeRepository: findByMovie(), findActive(), insert(), updateStatus().	config/ database.php	ShowtimeController

models/User.php	User entity + UserRepository: findByUsername(), findById(), insert(), toggleActive().	config/ database.php	AuthService, UserController
services/ BarcodeService.php	generateBarcode(): produces 'CS' + 12 uppercase hex chars from random bytes or time-based hash. Checks uniqueness against bookings.	Booking model	BookingController
services/AuthService.php	authenticate($username,$password): queries User model, compares password (plain in v1), returns user array or false.	User model	AuthController
services/ ReportService.php	calculateRevenue($filters): applies GROUP BY queries, formats results for report view.	Booking model	ReportController
assets/js/seatmap.js	Renders seat grid from JSON, handles click-to-select toggling, enforces single-seat selection, submits selected seat to booking form.	SeatController (AJAX)	map.php
assets/js/scanner.js	Listens for barcode input (keyup+Enter), POSTs to BookingController::scan(), displays success/error modal.	BookingController	scan.php

5. Database Documentation

5.1 Database Overview

Database Name: cinescan | Engine: MariaDB 10.4.32 | Character Set: utf8 / utf8mb4 | Collation: utf8_general_ci

The schema consists of 6 tables implementing a cinema booking lifecycle. Referential integrity is enforced via InnoDB foreign keys. All primary keys use AUTO_INCREMENT.

5.2 Table: users

Column	Type	Null	Key	Notes
ID	int(100)	NO	PK	AUTO_INCREMENT. Primary identifier.
Name	varchar(100)	NO		Full display name of the user.
Email	varchar(100)	NO		Email address. Not enforced as unique in schema.
Phone	varchar(20)	NO		Default 'N/A'.
Username	varchar(50)	NO		Login credential. Default empty string.
Password	varchar(100)	NO		Stored plain-text in v1 (see Security section). Max 100 chars.
Role	varchar(20)	YES		'Admin' or 'Staff'. Governs feature access.
Is_Active	tinyint(1)	YES		1 = active account, 0 = disabled. Default 1.

Seed data: Walk-in Customer (ID 1), Brent (ID 2, inactive), Administrator (ID 3, admin/admin123), Staff User (ID 4, staff/staff123).

5.3 Table: movies

Column	Type	Null	Key	Notes
ID	int(100)	NO	PK	AUTO_INCREMENT.
Title	varchar(100)	NO		Movie title (e.g. 'Spiderman').
Genre	varchar(50)	NO		Comma-separated genres (e.g. 'Action, Superpower').
Duration	int(10)	NO		Legacy field (minutes). Superseded by Duration_Mins.
Rating	varchar(10)	NO		Content rating: 'PG', 'R18', 'G', etc.
Poster	varchar(255)	YES		File path to poster image stored in assets/posters/.
Showtime	varchar(50)	YES		Legacy text field. Superseded by showtimes table.
Status	varchar(20)	YES		'Now Showing' or 'Ended'. Default 'Now Showing'.

Duration_Mins	int(11)	NO		Actual runtime in minutes. Default 120.
Hall_ID	int(11)	YES	FK→halls	Legacy direct hall assignment (now done via showtimes).
Price	decimal(10,2)	YES		Base ticket price. Default 250.00.

5.4 Table: cinema_halls

Column	Type	Null	Key	Notes
ID	int(11)	NO	PK	AUTO_INCREMENT.
Name	varchar(50)	NO		Display name e.g. 'Cinema 1'.
Total_Seats	int(11)	NO		Capacity. All halls set to 30. Default 50.
Is_Active	tinyint(1)	YES		Admin toggle. Default 1.
IsActive	tinyint(1)	NO		Duplicate of Is_Active (schema inconsistency). Both default 1.

Note: Is_Active and IsActive are redundant columns — a schema cleanup opportunity.

5.5 Table: showtimes

Column	Type	Null	Key	Notes
ID	int(11)	NO	PK	AUTO_INCREMENT.
Movie_ID	int(11)	NO	FK→movies	References movies.ID.
Hall_ID	int(11)	NO	FK→cinema_halls	References cinema_halls.ID.
Start_Time	datetime	NO		Scheduled start of showing.
End_Time	datetime	NO		Calculated End_Time = Start_Time + Duration_Mins.
Status	varchar(20)	YES		'Scheduled' or 'Ended'. Default 'Scheduled'.

5.6 Table: seats

Column	Type	Null	Key	Notes
ID	int(100)	NO	PK	AUTO_INCREMENT. Seats 69–158 seeded.
Movie_ID	int(11)	YES	FK (UQ)	Part of unique key uq_seat_per_movie (Movie_ID, Seat_Number).
Showtime	datetime	NO		Legacy showtime column (0000-00-00 in seed). Now uses showtimes table.
Seat_Number	varchar(10)	NO		Format: [A-C][1-10] e.g. 'A1', 'C10'.
Is_Occupied	tinyint(1)	YES		0 = free, 1 = taken. Default 0.

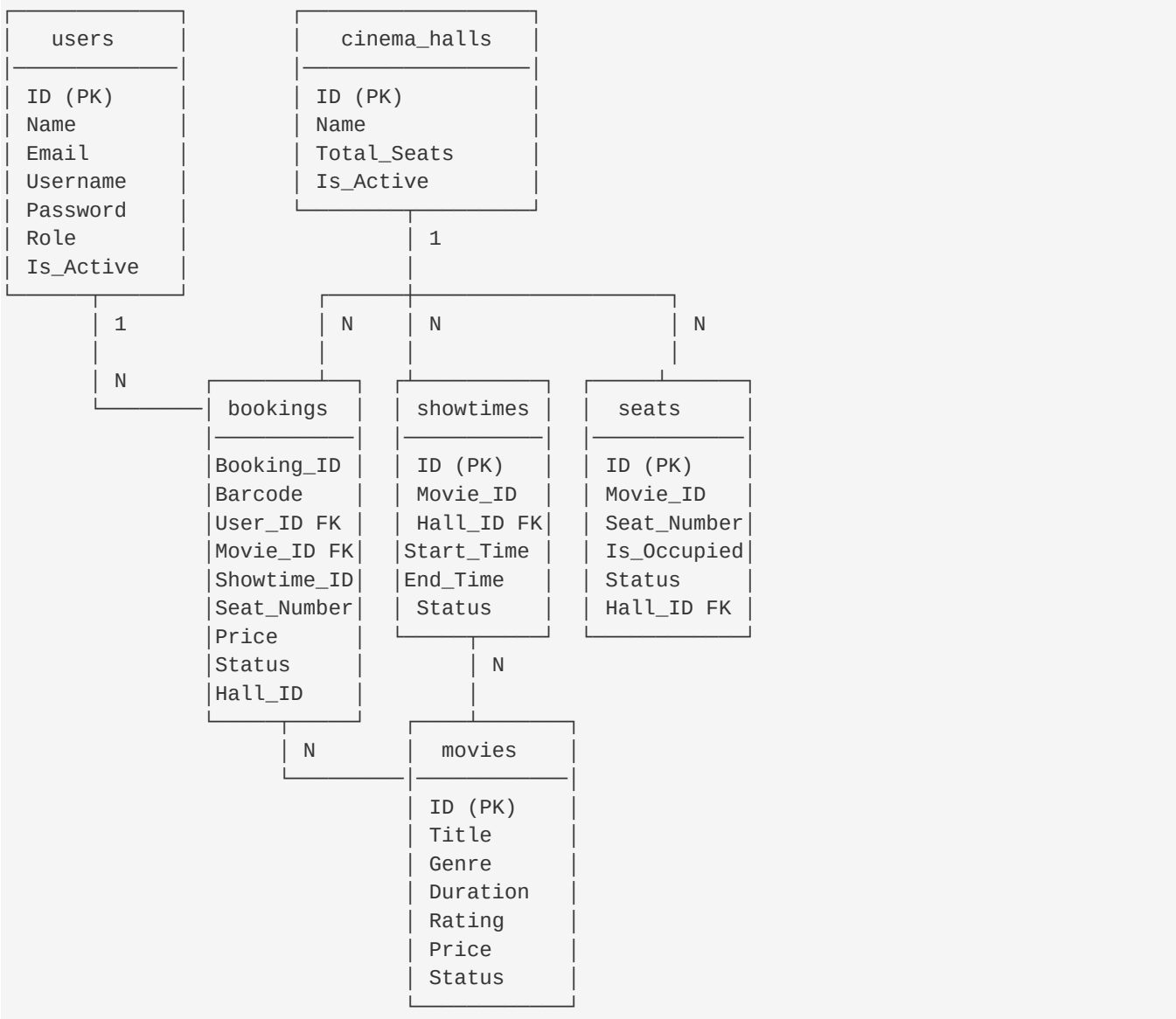
Status	varchar(20)	YES		'Available', 'Reserved', 'Occupied'. Default 'Available'.
Hall_ID	int(11)	NO	FK→cinema_halls	References cinema_halls.ID. Default 1.

5.7 Table: bookings

Column	Type	Null	Key	Notes
Booking_ID	int(255)	NO	PK	AUTO_INCREMENT. Unique booking identifier.
Barcode	varchar(100)	NO	UNIQUE	Format: 'CS' + 12 hex chars e.g. 'CSD1500C188294'. Enforced unique.
User_ID	int(100)	NO	FK→users	Staff member who created the booking.
Movie_ID	int(100)	NO	FK→movies	References movies.ID.
Showtime	datetime	NO		Denormalized showtime copy for quick display.
Seat_Number	varchar(10)	NO		Selected seat e.g. 'A1'.
Price	decimal(10,2)	NO		Price locked at time of booking.
Status	varchar(20)	YES		'Active' (booked), 'Used' (scanned), 'Cancelled'. Default 'ACTIVE'.
Showtime_ID	int(11)	YES		References showtimes.ID (nullable for legacy rows).
Hall_ID	int(11)	YES		Denormalized for performance/display.
Customer_Name	varchar(100)	YES		Walk-in name or reserved customer. Default empty.

5.8 Entity Relationship Diagram (ERD)

The following ASCII ERD shows all six tables and their relationships:



5.9 Relationship Analysis

Relationship	Cardinality	Description
users → bookings	1:N	One staff member creates many bookings. bookings.User_ID → users.ID
movies → bookings	1:N	One movie has many bookings. bookings.Movie_ID → movies.ID
movies → showtimes	1:N	One movie can have many scheduled showtimes. showtimes.Movie_ID → movies.ID
cinema_halls → showtimes	1:N	One hall hosts many showtimes. showtimes.Hall_ID → cinema_halls.ID
cinema_halls → seats	1:N	One hall contains many seats. seats.Hall_ID → cinema_halls.ID

movies → seats	0:N (UQ)	Unique constraint (Movie_ID, Seat_Number) prevents duplicate seat per movie-level reservation.

5.10 CRUD Mapping

Table	CREATE	READ	UPDATE	DELETE
users	UserController::store()	UserController::index(), AuthService	UserController::update()	UserController::toggleActive()
movies	MovieController::store()	MovieController::index(), show()	MovieController::update()	MovieController::destroy()
cinema_halls	HallController::store()	HallController::index()	HallController::update()	HallController::delete()
showtimes	ShowtimeController::store()	ShowtimeController::index()	ShowtimeController::updateStatus()	ShowtimeController::destroy()
seats	SeedSQL / HallController	SeatController::getMap()	BookingController (reserve/release)	—
bookings	BookingController::store()	BookingController::scan(), ReportController	BookingController::markUsed()	BookingController::cancel()

6. User Interface Documentation

6.1 Login Screen

Inputs	Username (text), Password (password)
Outputs	Redirect to Dashboard (success) or error message (failure)
Buttons	Login
Events	Form submit → AuthController::login()
Validations	Both fields required; error shown if credentials invalid or account inactive
Navigation	→ Admin Dashboard (Admin role) | → Staff Dashboard (Staff role)

6.2 Dashboard

Inputs	None (read-only overview)
Outputs	Summary cards: Total Movies, Active Bookings, Today's Revenue, Available Halls
Buttons	Nav links to all modules
Events	Page load → queries each table for counts
Validations	Role-based: Admin sees User Management; Staff does not
Navigation	→ Movies | Halls | Showtimes | Seat Map | Reports | Users (Admin)

6.3 Movie Management

Inputs	Title, Genre, Duration, Rating, Price, Poster file, Hall, Status
Outputs	Movie list table, success/error messages
Buttons	Add Movie, Edit, Delete/End
Events	Add: POST MovieController::store(); Edit: PUT update(); Delete: sets Status='Ended'
Validations	Title required, Duration > 0, Price ≥ 0, Rating must be selected
Navigation	← Dashboard | → Edit Movie form

6.4 Showtime Scheduler

Inputs	Movie (dropdown), Hall (dropdown), Start Date/Time, Duration (auto-calculates End_Time)
Outputs	Showtime list with status badges
Buttons	Add Showtime, End Showtime, Delete
Events	Add: POST ShowtimeController::store(); status update on End click
Validations	No hall overlap (same hall, overlapping time), movie must be 'Now Showing'
Navigation	← Dashboard | → Seat Map for a showtime

6.5 Seat Map

Inputs	Showtime selected (from URL param). Click on seat.
Outputs	3×10 grid. Color legend: Green=Available, Red=Reserved, Gray=Used/Occupied
Buttons	Proceed to Book (enabled after seat selection)
Events	Page load: AJAX GET SeatController::getMap() → JS renders grid; Click seat → toggle selection
Validations	Only one seat selectable; reserved seats non-clickable
Navigation	← Showtime list | → Booking Form

6.6 Booking Form

Inputs	Customer_Name (optional), confirms Movie, Showtime, Seat, Price
Outputs	Booking record created, unique Barcode generated, Ticket view
Buttons	Confirm & Book, Cancel
Events	POST BookingController::store() → BarcodeService::generateBarcode() → INSERT bookings → UPDATE seats
Validations	Seat re-validated at save time to prevent race conditions
Navigation	← Seat Map | → Ticket / Receipt view

6.7 Ticket / Receipt

Inputs	Booking_ID (auto-loaded)
Outputs	Barcode display, movie info, seat, hall, time, price, customer name
Buttons	Print, Back to Dashboard
Events	Print: window.print()
Validations	None (read-only)
Navigation	← Booking Form | → Dashboard

6.8 Barcode Scanner

Inputs	Barcode string (USB scanner or keyboard)
Outputs	Success modal (booking marked Used) or Error modal (invalid/already used)
Buttons	Scan (or press Enter), Clear
Events	keyup Enter → AJAX POST BookingController::scan() → UPDATE bookings SET Status='Used'
Validations	Barcode must exist and have Status='Active'
Navigation	← Dashboard

6.9 Reports

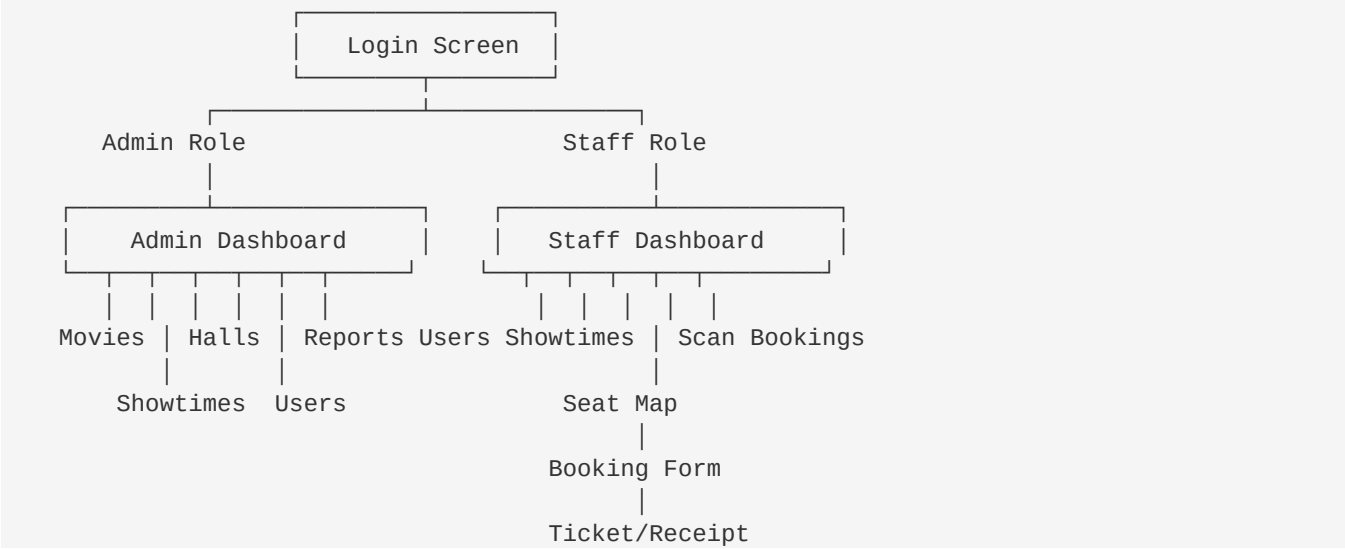
Inputs	Filter: Date range, Movie, Hall, Status

Outputs	Filtered bookings table, total revenue sum, seat occupancy %
Buttons	Filter, Export CSV, Reset
Events	Filter submit → GET ReportController::index($filters)
Validations	Date range: start ≤ end
Navigation	← Dashboard

6.10 User Management (Admin)

Inputs	Name, Email, Phone, Username, Password, Role, Is_Active toggle
Outputs	User list with status badges
Buttons	Add User, Edit, Deactivate
Events	POST UserController::store(); PUT update(); PATCH toggleActive()
Validations	Username unique check, Role must be 'Admin' or 'Staff'
Navigation	← Dashboard

6.11 Navigation Flow Diagram



7. Complete Function and Method Documentation

login() — AuthController.php

Purpose	Entry point for authentication. Reads POST username/password.
Parameters	$_POST['username'], $_POST['password']
Returns	Redirect to dashboard or re-render login with error
Dependencies	AuthService::authenticate()
Internal Logic	1. Sanitize inputs 2. Call AuthService::authenticate() 3. If true → set $_SESSION[user_id, role, name] → redirect 4. Else → flash error → reload login
Called By	login.php form submit
Calls To	AuthService::authenticate()

logout() — AuthController.php

Purpose	Destroys the user session and redirects to login.
Parameters	None
Returns	Redirect to /login
Dependencies	PHP session_destroy()
Internal Logic	1. session_start() 2. session_unset() 3. session_destroy() 4. header('Location: /login')
Called By	Nav logout link
Calls To	None

authenticate($username, $password) — AuthService.php

Purpose	Validates login credentials against the users table.
Parameters	$username: string, $password: string
Returns	User array on success | false on failure
Dependencies	UserRepository::findByUsername()
Internal Logic	1. Call UserRepository::findByUsername($username) 2. If no record → return false 3. If Is_Active == 0 → return false 4. Compare $password with stored Password 5. Return user array if match, false if not
Called By	AuthController::login()
Calls To	UserRepository::findByUsername()

findByUsername($username) — models/User.php

Purpose	Fetches a single user record by username.
Parameters	$username: string
Returns	Associative array of user row | null
Dependencies	PDO/mysqli connection

Internal Logic	SELECT * FROM users WHERE Username = ? AND Is_Active = 1 LIMIT 1
Called By	AuthService::authenticate()
Calls To	DB connection

findById($id) — models/User.php

Purpose	Fetches a user by primary key.
Parameters	$id: int
Returns	User array | null
Dependencies	DB connection
Internal Logic	SELECT * FROM users WHERE ID = ? LIMIT 1
Called By	Various controllers
Calls To	DB connection

insert($data) — models/User.php

Purpose	Inserts a new user record.
Parameters	$data: array [Name, Email, Phone, Username, Password, Role]
Returns	Last insert ID | false
Dependencies	DB connection
Internal Logic	INSERT INTO users (Name,Email,Phone,Username,Password,Role) VALUES (?,?,?,?,?,?)
Called By	UserController::store()
Calls To	DB connection

toggleActive($id) — models/User.php

Purpose	Flips Is_Active between 0 and 1.
Parameters	$id: int
Returns	Boolean success
Dependencies	DB connection
Internal Logic	UPDATE users SET Is_Active = 1 - Is_Active WHERE ID = ?
Called By	UserController::toggleActive()
Calls To	DB connection

findAll() — models/Movie.php

Purpose	Returns all movies regardless of status.
Parameters	None
Returns	Array of movie rows

Dependencies	DB connection
Internal Logic	SELECT * FROM movies ORDER BY ID DESC
Called By	MovieController::index()
Calls To	DB connection

findById($id) — models/Movie.php

Purpose	Returns a single movie by ID.
Parameters	$id: int
Returns	Movie array | null
Dependencies	DB connection
Internal Logic	SELECT * FROM movies WHERE ID = ? LIMIT 1
Called By	ShowtimeController, BookingController
Calls To	DB connection

insert($data) — models/Movie.php

Purpose	Inserts a new movie record.
Parameters	$data: array [Title,Genre,Duration,Rating,Poster,Status,Duration_Mins,Hall_ID,Price]
Returns	Last insert ID | false
Dependencies	DB connection
Internal Logic	INSERT INTO movies (...) VALUES (...)
Called By	MovieController::store()
Calls To	DB connection

update($id, $data) — models/Movie.php

Purpose	Updates all editable fields of a movie.
Parameters	$id: int, $data: array
Returns	Boolean success
Dependencies	DB connection
Internal Logic	UPDATE movies SET Title=?,Genre=?,... WHERE ID=?
Called By	MovieController::update()
Calls To	DB connection

updateStatus($id, $status) — models/Movie.php

Purpose	Sets Status field (Now Showing / Ended).
Parameters	$id: int, $status: string
Returns	Boolean success

Dependencies	DB connection
Internal Logic	UPDATE movies SET Status=? WHERE ID=?
Called By	MovieController::destroy(), ShowtimeController
Calls To	DB connection

findByMovie($movieId) — models/Showtime.php

Purpose	Returns all showtimes for a given movie.
Parameters	$movieId: int
Returns	Array of showtime rows
Dependencies	DB connection
Internal Logic	SELECT s.*, h.Name AS HallName FROM showtimes s JOIN cinema_halls h ON s.Hall_ID=h.ID WHERE s.Movie_ID=? ORDER BY s.Start_Time
Called By	ShowtimeController::index()
Calls To	DB connection

findActive() — models/Showtime.php

Purpose	Returns showtimes with Status='Scheduled' and End_Time > NOW().
Parameters	None
Returns	Array of active showtimes
Dependencies	DB connection
Internal Logic	SELECT * FROM showtimes WHERE Status='Scheduled' AND End_Time > NOW()
Called By	DashboardController, SeatController
Calls To	DB connection

insert($data) — models/Showtime.php

Purpose	Creates a new showtime after overlap validation.
Parameters	$data: array [Movie_ID, Hall_ID, Start_Time, End_Time]
Returns	Last insert ID | false
Dependencies	DB connection
Internal Logic	INSERT INTO showtimes (Movie_ID, Hall_ID, Start_Time, End_Time, Status) VALUES (?,?,?,?,'Scheduled')
Called By	ShowtimeController::store()
Calls To	DB connection

checkOverlap($hallId, $start, $end, $excludeId) — models/Showtime.php

Purpose	Checks if a hall is already occupied for a given time range.
Parameters	$hallId: int, $start: datetime, $end: datetime, $excludeId: int

Returns	int (count of conflicts)
Dependencies	DB connection
Internal Logic	SELECT COUNT(*) FROM showtimes WHERE Hall_ID=? AND Status='Scheduled' AND NOT (End_Time<=? OR Start_Time>=?) AND ID != ?
Called By	ShowtimeController::store()
Calls To	DB connection

findByHall($hallId) — models/Seat.php

Purpose	Returns all seats belonging to a hall.
Parameters	$hallId: int
Returns	Array of seat rows
Dependencies	DB connection
Internal Logic	SELECT * FROM seats WHERE Hall_ID=? ORDER BY Seat_Number
Called By	SeatController::getMap()
Calls To	DB connection

reserve($seatId) — models/Seat.php

Purpose	Marks a seat as Reserved.
Parameters	$seatId: int
Returns	Boolean success
Dependencies	DB connection
Internal Logic	UPDATE seats SET Is_Occupied=1, Status='Reserved' WHERE ID=?
Called By	BookingController::store()
Calls To	DB connection

release($seatId) — models/Seat.php

Purpose	Returns a seat to Available (on booking cancel).
Parameters	$seatId: int
Returns	Boolean success
Dependencies	DB connection
Internal Logic	UPDATE seats SET Is_Occupied=0, Status='Available' WHERE ID=?
Called By	BookingController::cancel()
Calls To	DB connection

isOccupied($hallId, $seatNumber) — models/Seat.php

Purpose	Quick check if a specific seat is taken.

Parameters	$hallId: int, $seatNumber: string
Returns	Boolean
Dependencies	DB connection
Internal Logic	SELECT Is_Occupied FROM seats WHERE Hall_ID=? AND Seat_Number=? LIMIT 1
Called By	BookingController::store() (race guard)
Calls To	DB connection

insert($data) — models/Booking.php

Purpose	Inserts a new booking record.
Parameters	$data: array [Barcode, User_ID, Movie_ID, Showtime, Seat_Number, Price, Showtime_ID, Hall_ID, Customer_Name]
Returns	Last insert ID | false
Dependencies	DB connection
Internal Logic	INSERT INTO bookings (...) VALUES (...)
Called By	BookingController::store()
Calls To	DB connection

findByBarcode($barcode) — models/Booking.php

Purpose	Looks up a booking by its scanned barcode.
Parameters	$barcode: string
Returns	Booking array | null
Dependencies	DB connection
Internal Logic	SELECT * FROM bookings WHERE Barcode=? LIMIT 1
Called By	BookingController::scan()
Calls To	DB connection

updateStatus($bookingId, $status) — models/Booking.php

Purpose	Changes booking lifecycle status.
Parameters	$bookingId: int, $status: string
Returns	Boolean success
Dependencies	DB connection
Internal Logic	UPDATE bookings SET Status=? WHERE Booking_ID=?
Called By	BookingController::markUsed(), cancel()
Calls To	DB connection

findByShowtime($showtimeId) — models/Booking.php

Purpose	Returns all bookings for a specific showtime.
Parameters	$showtimeId: int
Returns	Array of booking rows
Dependencies	DB connection
Internal Logic	SELECT * FROM bookings WHERE Showtime_ID=? AND Status != 'Cancelled'
Called By	SeatController::getMap() (overlay)
Calls To	DB connection

generateBarcode() — services/BarcodeService.php

Purpose	Generates a unique ticket barcode for a new booking.
Parameters	None
Returns	string: 'CS' + 12 uppercase hex chars
Dependencies	BookingRepository::findByBarcode()
Internal Logic	1. Generate 6 random bytes via random_bytes(6) or uniqid 2. Convert to 12-char uppercase hex 3. Prepend 'CS' 4. Check uniqueness via findByBarcode() 5. If collision, regenerate 6. Return unique barcode
Called By	BookingController::store()
Calls To	BookingRepository::findByBarcode()

store() — controllers/BookingController.php

Purpose	Core booking creation: validates seat, generates barcode, saves booking.
Parameters	POST: showtime_id, seat_number, hall_id, movie_id, customer_name
Returns	Redirect to ticket view | error flash
Dependencies	BarcodeService, SeatRepository, BookingRepository
Internal Logic	1. Validate POST params 2. Re-check seat availability (isOccupied) 3. Compute price from movie 4. Call BarcodeService::generateBarcode() 5. INSERT booking 6. UPDATE seat status to Reserved 7. Redirect to ticket view
Called By	booking form submit
Calls To	BarcodeService, SeatRepository, BookingRepository

scan() — controllers/BookingController.php

Purpose	Processes barcode scan at cinema entry gate.
Parameters	POST: barcode string
Returns	JSON: {success, message, booking} or {error}
Dependencies	BookingRepository
Internal Logic	1. findByBarcode($barcode) 2. If null → JSON error 'Not Found' 3. If Status=='Used' → JSON error 'Already Used' 4. If Status=='Cancelled' → JSON error 5. updateStatus(id, 'Used') 6. Return JSON success with booking details

Called By	scanner.js AJAX
Calls To	BookingRepository::findByBarcode(), updateStatus()

cancel($id) — controllers/BookingController.php

Purpose	Cancels an active booking and releases the seat.
Parameters	POST: booking_id
Returns	Redirect with flash message
Dependencies	BookingRepository, SeatRepository
Internal Logic	1. findById($id) 2. Check status == 'Active' 3. updateStatus($id, 'Cancelled') 4. SeatRepository::release($seatId) 5. Redirect
Called By	Booking list cancel button
Calls To	BookingRepository, SeatRepository

getMap($showtimeId) — controllers/SeatController.php

Purpose	Builds seat availability grid for the seat map UI.
Parameters	$showtimeId: int
Returns	JSON array of seat objects {seat_number, status}
Dependencies	ShowtimeRepository, SeatRepository, BookingRepository
Internal Logic	1. findById showtime 2. Get hall_id from showtime 3. SeatRepository::findByHall(hall_id) 4. BookingRepository::findByShowtime(showtime_id) 5. Build array: for each seat, if in bookings list mark 'Reserved' or 'Used' else 'Available' 6. Return JSON
Called By	seatmap.js AJAX GET
Calls To	ShowtimeRepository, SeatRepository, BookingRepository

index($filters) — controllers/ReportController.php

Purpose	Fetches and aggregates booking data for the reports screen.
Parameters	GET: date_from, date_to, movie_id, hall_id, status
Returns	Rendered reports view with data
Dependencies	ReportService
Internal Logic	1. Parse filter params 2. Call ReportService::getBookings($filters) 3. Call ReportService::getTotalRevenue($filters) 4. Pass to view
Called By	Reports nav / form submit
Calls To	ReportService

getBookings($filters) — services/ReportService.php

Purpose	Returns filtered bookings joined with movie and hall info.
Parameters	$filters: array
Returns	Array of booking rows with joined names

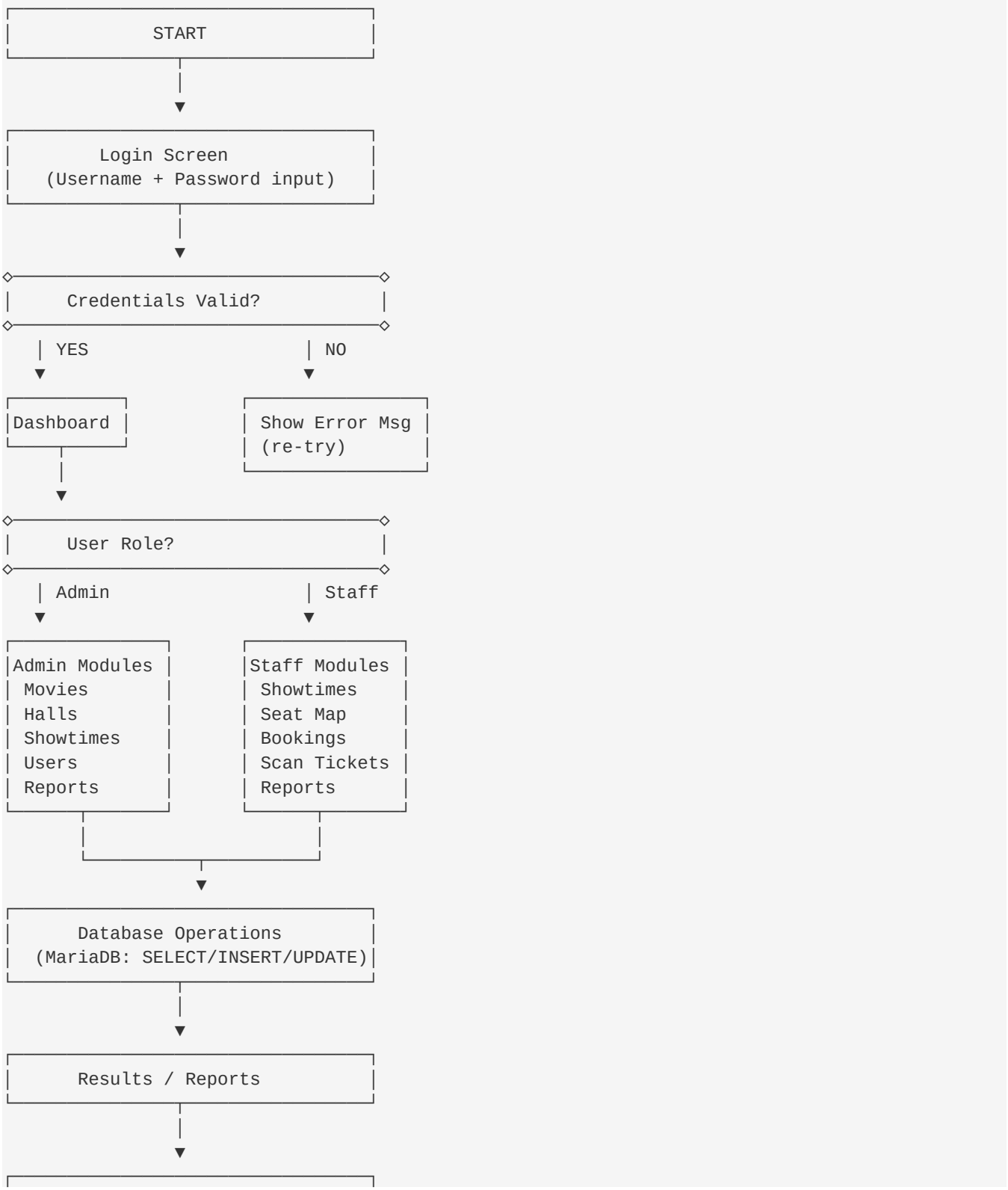
Dependencies	BookingRepository
Internal Logic	SELECT b.*, m.Title, h.Name FROM bookings b JOIN movies m ON b.Movie_ID=m.ID JOIN cinema_halls h ON b.Hall_ID=h.ID WHERE [filter conditions] ORDER BY b.Booking_ID DESC
Called By	ReportController::index()
Calls To	DB connection

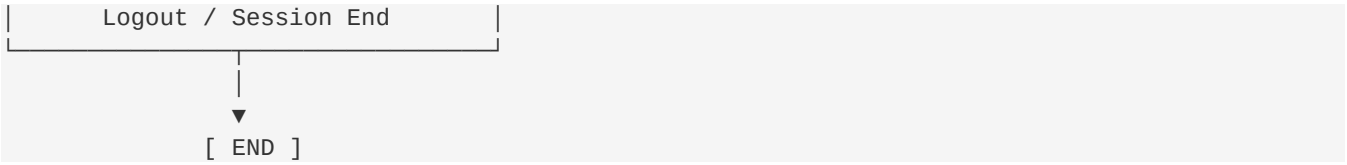
getTotalRevenue($filters) — services/ReportService.php

Purpose	Calculates sum of ticket prices for filtered bookings.
Parameters	$filters: array
Returns	float: total revenue
Dependencies	DB connection
Internal Logic	SELECT SUM(Price) FROM bookings WHERE Status != 'Cancelled' AND [filter conditions]
Called By	ReportController::index()
Calls To	DB connection

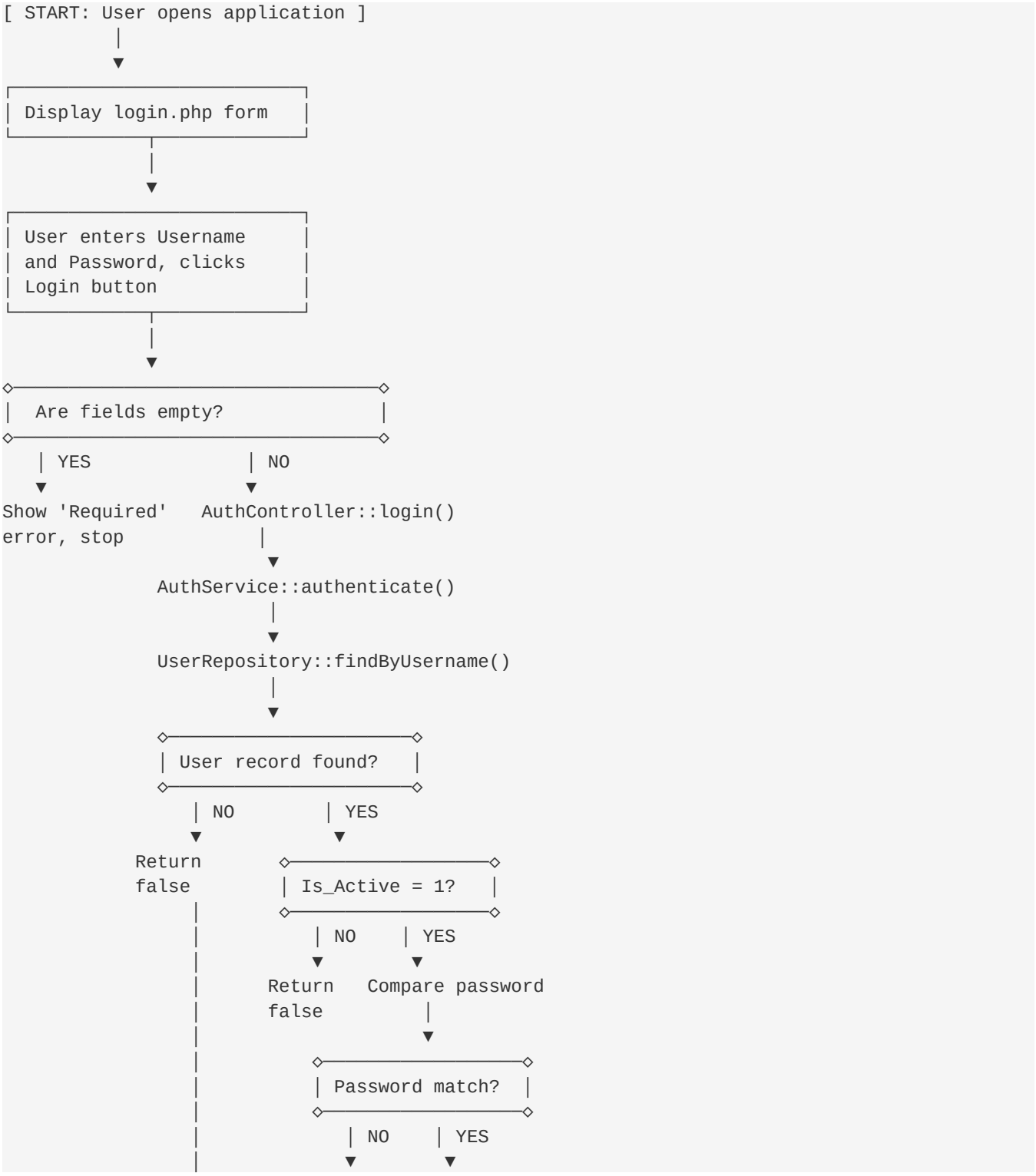
8. Complete System Flowchart Analysis

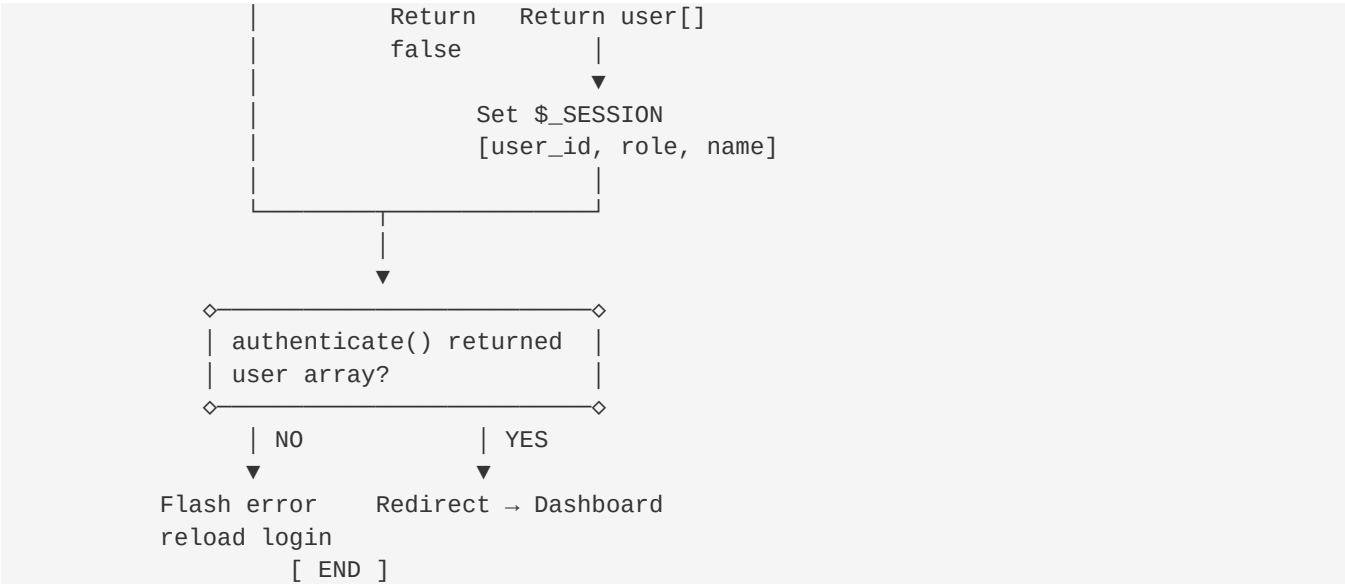
8.1 Level 1 – System Flowchart



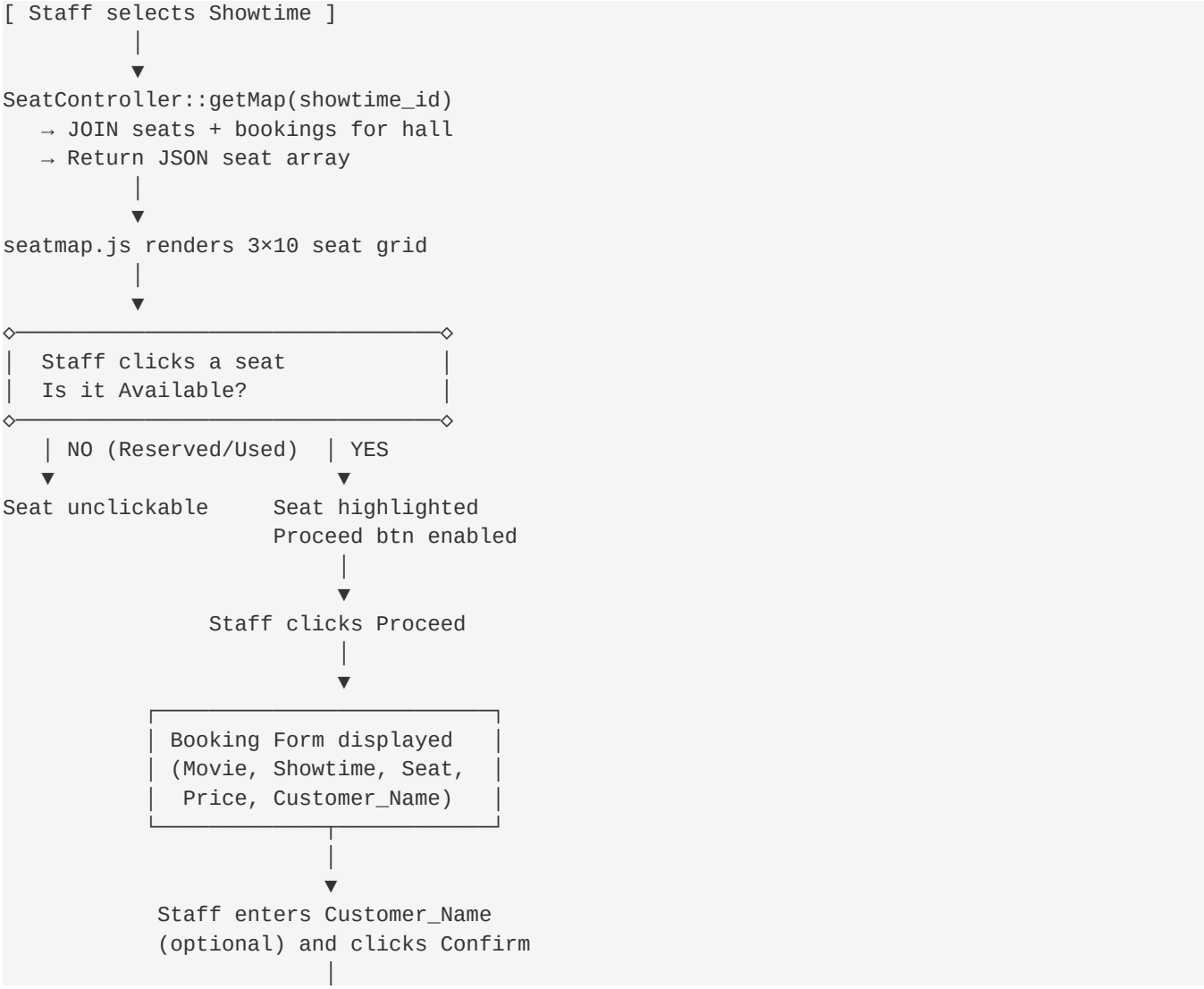


8.2 Level 2 – Login Module Flowchart





8.3 Level 2 – Booking Module Flowchart





8.4 Level 2 – Barcode Scan Module Flowchart



```
                                Return JSON success
                                Show green modal
                                (Movie, Seat, Hall info)
                                     [ END ]
```

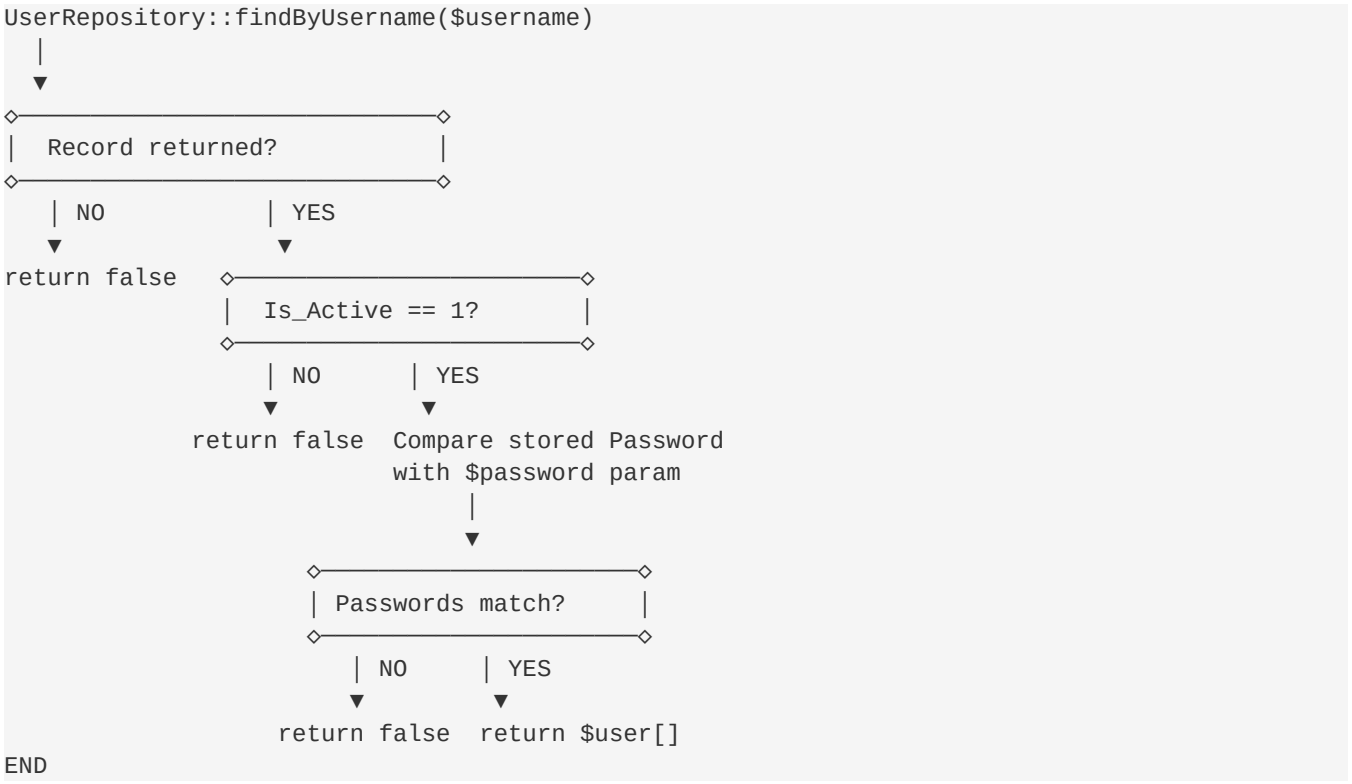
8.5 Level 2 – Reporting Module Flowchart

```
[ Admin/Staff opens Reports ]
          |
          ▼
ReportController::index(GET params)
          |
          ▼
Parse filter params: date_from, date_to,
movie_id, hall_id, status
          |
          ▼
ReportService::getBookings($filters)
  → SELECT b.*, m.Title, h.Name
     FROM bookings b
     JOIN movies m ON b.Movie_ID=m.ID
     JOIN cinema_halls h ON b.Hall_ID=h.ID
     WHERE [filter conditions]
          |
          ▼
ReportService::getTotalRevenue($filters)
  → SELECT SUM(Price) FROM bookings
     WHERE Status != 'Cancelled'
     AND [filter conditions]
          |
          ▼
Render reports/index.php view
  → Display filtered booking table
  → Show total revenue
  → Show occupancy % per hall
          |
          ▼
◇──────────────────────────────────◇
|  Export CSV requested?          |
◇──────────────────────────────────◇
   | YES              | NO
   ▼                  ▼
Generate CSV       Display in browser
headers + data     only
Force download
        [ END ]
```

8.6 Level 3 – File-Level Flowcharts

AuthService.php

```
START: authenticate($username, $password) called
  |
  ▼
```



BarcodeService.php



SeatController.php – getMap()



```
  ▼
Extract $hall_id from showtime
  │
  ▼
SeatRepository::findByHall($hall_id)
→ Returns 30 seat rows for hall
  │
  ▼
BookingRepository::findByShowtime($showtime_id)
→ Returns active/used bookings for this show
  │
  ▼
Build $bookedSeats map: [Seat_Number => Status]
  │
  ▼
FOR EACH seat in hall seats:
  │
  ▼
◇──────────────────────────────────◇
│  seat_number in $bookedSeats?    │
◇──────────────────────────────────◇
   │ YES                  │ NO
   ▼                      ▼
seat.status =        seat.status = 'Available'
$bookedSeats[seatNum]
  │                        │
  └────────────┬───────────┘
               ▼
Append seat to JSON output array
  │
  ▼
Return JSON array to seatmap.js
END
```

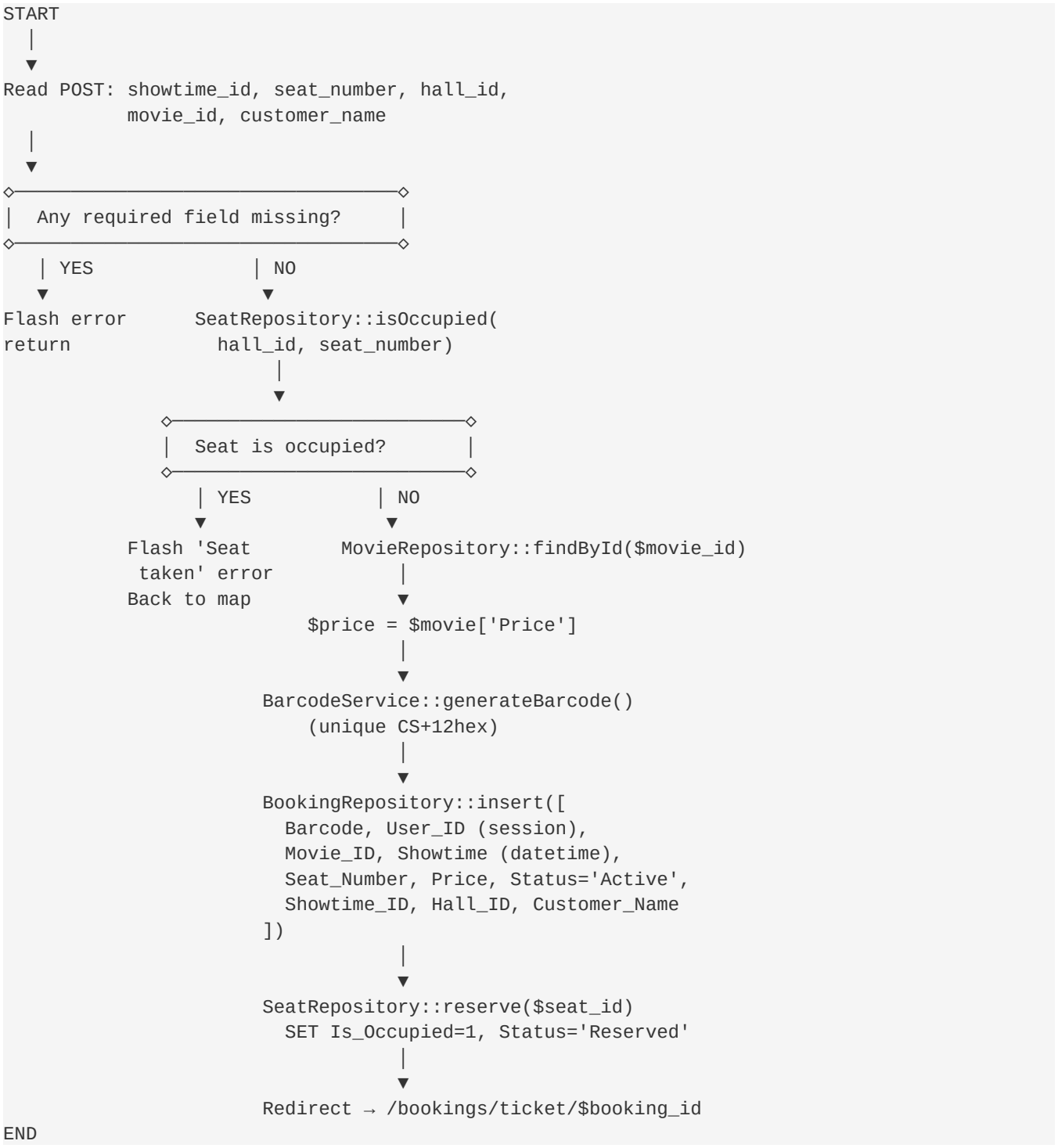
ShowtimeController.php – store()

```
START: POST /showtimes/store
  │
  ▼
Read: movie_id, hall_id, start_time, duration_mins
  │
  ▼
Calculate end_time = start_time + duration_mins
  │
  ▼
ShowtimeRepository::checkOverlap(hall_id, start, end)
  │
  ▼
◇──────────────────────────────────◇
│  Overlap count > 0?              │
◇──────────────────────────────────◇
   │ YES                  │ NO
   ▼                      ▼
Flash error           ShowtimeRepository::insert()
'Hall already             │
```

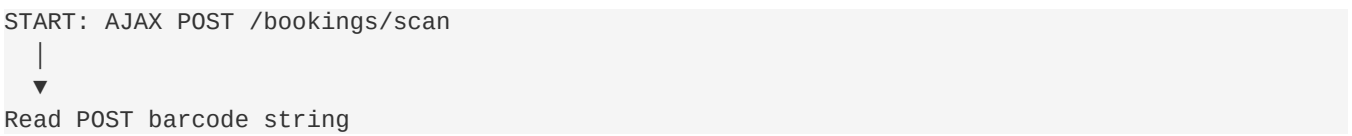
```
 occupied'                ▼
Redirect back       Flash success
                    Redirect to showtime list
END
```

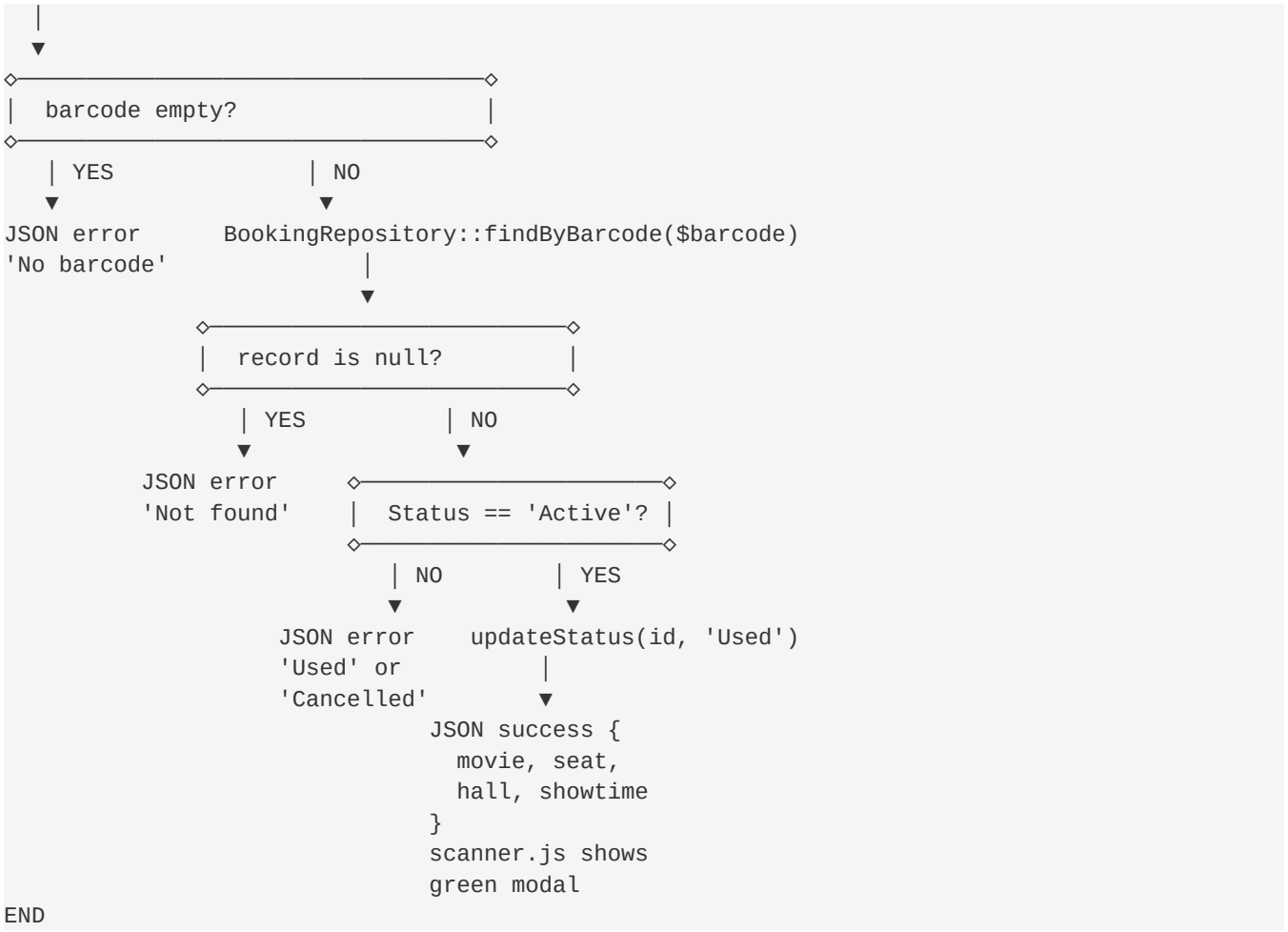
8.7 Level 4 – Function-Level Flowcharts

BookingController::store() – Detailed



BookingController::scan() – Detailed




barcode empty?
YES
NO
JSON error
'No barcode'
BookingRepository::findByBarcode($barcode)
record is null?
YES
NO
JSON error
'Not found'
Status == 'Active'?
NO
YES
JSON error
'Used' or
'Cancelled'
updateStatus(id, 'Used')
JSON success {
movie, seat,
hall, showtime
}
scanner.js shows
green modal
END

8.8 Level 5 – System Dependency / Call Chain

```
PRESENTATION LAYER (Views + JS)
  login.php ──────────────────────► AuthController::login()
  seatmap.js (AJAX) ──────────────► SeatController::getMap()
  scanner.js (AJAX) ──────────────► BookingController::scan()
  booking/create.php ─────────────► BookingController::store()
  reports/index.php ──────────────► ReportController::index()
  users/index.php ────────────────► UserController::*()

CONTROLLER LAYER
  AuthController::login()
    └──► AuthService::authenticate()
           └──► UserRepository::findByUsername()
                  └──► DB: SELECT users

  BookingController::store()
    ├──► SeatRepository::isOccupied()     ──► DB: SELECT seats
    ├──► MovieRepository::findById()      ──► DB: SELECT movies
    ├──► BarcodeService::generateBarcode()
    │       └──► BookingRepository::findByBarcode() ──► DB: SELECT bookings
    ├──► BookingRepository::insert()      ──► DB: INSERT bookings
    └──► SeatRepository::reserve()        ──► DB: UPDATE seats

  BookingController::scan()
    ├──► BookingRepository::findByBarcode() ──► DB: SELECT bookings
    └──► BookingRepository::updateStatus()  ──► DB: UPDATE bookings

  SeatController::getMap()
    ├──► ShowtimeRepository::findById()    ──► DB: SELECT showtimes
    ├──► SeatRepository::findByHall()      ──► DB: SELECT seats
    └──► BookingRepository::findByShowtime() ─► DB: SELECT bookings

  ShowtimeController::store()
    ├──► ShowtimeRepository::checkOverlap() ─► DB: SELECT showtimes COUNT
    └──► ShowtimeRepository::insert()       ─► DB: INSERT showtimes

  ReportController::index()
    ├──► ReportService::getBookings()       ─► DB: SELECT JOIN
    └──► ReportService::getTotalRevenue()   ─► DB: SELECT SUM

DATABASE LAYER (MariaDB)
  Tables: users, movies, cinema_halls, showtimes, seats, bookings
```

9. Algorithm Analysis

Barcode Generation

Objective	Generate a globally unique ticket identifier with low collision probability.
Inputs	None (uses cryptographic random source)
Processing Steps	1. Call random_bytes(6) → 6 cryptographically random bytes 2. bin2hex() → 12 lowercase hex chars 3. strtoupper() → uppercase 4. Prepend 'CS' → 14-char barcode 5. Query bookings table for existence 6. If found → repeat from step 1 7. Return barcode
Outputs	14-character string: 'CS' + 12 hex chars. E.g. 'CSD1500C188294'
Complexity	O(1) average. Collision probability: 1 / (16^12) ≈ 1 in 281 trillion per attempt.
Optimization	The uniqueness check adds one DB query per booking. For extremely high volume, switch to UUID v4 (guaranteed unique) or sequence-based IDs.

Seat Availability Resolution

Objective	Determine real-time status of each seat for a specific showtime.
Inputs	showtime_id → derives hall_id; All seats for hall; All active bookings for showtime
Processing Steps	1. Load all N seats for hall (N=30) 2. Load all M bookings for showtime 3. Build O(1) hash map: {seat_number → booking_status} 4. For each seat: if seat_number in map → apply booking status; else → 'Available' 5. Return array of 30 seat objects with status
Outputs	JSON array: [{seat_number, status}] × 30
Complexity	O(N + M) time, O(M) space. With N=30 and M≤30, effectively O(1).
Optimization	Add Redis cache keyed by showtime_id; invalidate on booking INSERT/UPDATE. Would reduce DB load at peak.

Hall Overlap Detection

Objective	Prevent scheduling two movies in the same hall at the same time.
Inputs	hall_id, proposed_start, proposed_end, exclude_id (for edit)
Processing Steps	1. Query showtimes WHERE Hall_ID=? AND Status='Scheduled' 2. Exclude current record (for updates) 3. Apply interval overlap formula: NOT (existing.End <= new.Start OR existing.Start >= new.End) 4. COUNT(*) > 0 indicates conflict
Outputs	Integer count: 0 = no conflict, >0 = conflict exists
Complexity	O(S) where S = number of scheduled showtimes. DB index on Hall_ID makes this near O(log S).
Optimization	Add composite index on (Hall_ID, Start_Time, End_Time) for faster range queries.

Revenue Aggregation

Objective	Sum ticket revenue across filtered booking records.
Inputs	Filter array: date_from, date_to, movie_id, hall_id, status
Processing Steps	1. Build dynamic WHERE clause from filter array 2. Execute SELECT SUM(Price)

	FROM bookings WHERE [conditions] 3. Exclude Cancelled bookings from sum 4. Return float value
Outputs	float: total revenue in PHP Peso (₱)
Complexity	O(B) where B = filtered booking count. Full table scan without index; indexing Status + Showtime would improve.
Optimization	Add index on (Status, Showtime) for date-range reports; consider materialized daily revenue summary table.

10. Security Analysis

Concern	Severity	Current State	Recommendation
Authentication	MEDIUM	Username/password login with role-based session. Sessions identify user_id and role.	Add CSRF token to login form. Enforce session timeout (e.g. 30 min idle). Use session_regenerate_id() after login.
Password Storage	CRITICAL	Passwords stored as plain text in users table (admin123, staff123 in seed data).	Immediately implement password_hash() with PASSWORD_BCRYPT on creation and password_verify() on login. Never store plain text.
SQL Injection	HIGH	If raw string concatenation is used in queries (e.g. $id from $_GET), all tables are at risk.	Use PDO prepared statements with named or positional placeholders for ALL queries. Never concatenate user input.
XSS	MEDIUM	Output of movie titles, customer names, barcodes must be escaped before display.	Apply htmlspecialchars($var, ENT_QUOTES, 'UTF-8') to all user-originated data in views.
Authorization	HIGH	Role check must be enforced on every admin-only controller method, not just nav links.	Add a requireRole('Admin') middleware/guard at the top of UserController and sensitive report actions.
CSRF	MEDIUM	Booking, scan, and user management POST actions are vulnerable without CSRF tokens.	Generate a per-session CSRF token; embed it as hidden input; validate on every state-changing POST.
Session Fixation	MEDIUM	Session ID not regenerated after login allows fixation attacks.	Call session_regenerate_id(true) immediately after successful authentication.
Brute Force	MEDIUM	No rate limiting on login attempts allows credential stuffing.	Add login attempt counter (per IP or username) with lockout after N failures (e.g. 5). Use CAPTCHA on repeated failure.
Barcode Predictability	LOW	Barcodes use random_bytes() which is cryptographically secure. Collision probability negligible.	Maintain current approach. Optionally add HMAC signing to detect forged barcodes.
File Upload	HIGH	Movie poster uploads (Poster field) may allow malicious file execution if not validated.	Whitelist allowed MIME types (image/jpeg, image/png), validate server-side with getimagesize(), store outside web root or rename with random UUID filename.

11. Installation and Deployment Guide

11.1 Software Requirements

Component	Version	Notes
Web Server	Apache 2.4+ or Nginx 1.18+	Tested on XAMPP / WAMP stack
PHP	8.2.x	Matches server PHP Version 8.2.12
Database	MariaDB 10.4.32 or MySQL 8.0+	InnoDB engine required for FK support
phpMyAdmin	5.2.1	Database administration (optional for production)
Browser	Chrome 100+ / Firefox 100+ / Edge 100+	For USB barcode scanner support (HID)
OS	Windows 10/11 or Linux (Ubuntu 20.04+)	Cross-platform compatible

11.2 Installation Steps

```
Step 1: Clone or extract the CineScan project folder into your web server root.
        XAMPP: C:\xampp\htdocs\cinescan\
        Linux: /var/www/html/cinescan/

Step 2: Start Apache and MariaDB services (XAMPP Control Panel or systemctl).

Step 3: Open phpMyAdmin at http://localhost/phpmyadmin
        Create a new database named: cinescan
        Character set: utf8 | Collation: utf8_general_ci

Step 4: Import the database schema:
        phpMyAdmin → cinescan → Import → Select cinescan.sql → Go
        OR via CLI: mysql -u root -p cinescan < cinescan.sql

Step 5: Configure database connection in config/database.php:
        define('DB_HOST', '127.0.0.1');
        define('DB_USER', 'root');
        define('DB_PASS', '');   // set your MariaDB root password
        define('DB_NAME', 'cinescan');

Step 6: Ensure the assets/posters/ directory is writable:
        chmod 755 assets/posters/   (Linux)

Step 7: Open browser → http://localhost/cinescan/
        Login with: admin / admin123  (Admin) or  staff / staff123  (Staff)

Step 8 (Production): Change default passwords immediately via User Management.
        Enable HTTPS via SSL certificate on the web server.
        Set PHP error_reporting(0) and log errors to file only.
```

12. Code Quality Assessment

Dimension	Rating	Assessment & Recommendation
Maintainability	**Moderate**	Separation into controllers/models/services/views provides good structure. However, mixing business logic with SQL in some controllers reduces cohesion. Recommend strict adherence to repository pattern.
Scalability	**Low-Medium**	Single-server architecture with no caching. Seat map AJAX works for small halls (30 seats). For 500+ seat venues, introduce Redis caching and pagination. Database lacks indexes on frequently filtered columns.
Modularity	**Good**	Each module (Auth, Movie, Booking, Showtime, Seat, Report, User) is independently addressable. BarcodeService is correctly isolated.
Readability	**Moderate**	Function names are descriptive. Lack of PHPDoc comments on methods reduces onboarding speed. Recommend JSDoc for JS files and PHPDoc for all classes.
Performance	**Moderate**	Small dataset (3 halls, ~10 movies) causes no issues. Real-world performance concerns: N+1 query risk in report generation; no connection pooling; seat map rebuilds from DB on every request.
Security	**LOW**	Critical: plain-text password storage. High: potential SQL injection if prepared statements are not universally used. See Section 10 for full remediation list.
Test Coverage	**None detected**	No unit tests, integration tests, or automated test suite found. Recommend PHPUnit for backend and Jest/Cypress for frontend.
Schema Design	**Good**	Normalized with FK constraints. Minor issue: duplicate Is_Active/IsActive column in cinema_halls. Legacy columns (movies.Showtime, seats.Showtime) add confusion.

13. Future Enhancements

Enhancement	Description
Security (Immediate)	Implement password_hash()/password_verify(), PDO prepared statements for all queries, CSRF tokens on all forms, and HTTPS enforcement.
Online Booking Portal	Add a customer-facing web or mobile interface for self-service seat selection, payment integration (GCash, PayMaya, credit card via Stripe/PayMongo), and e-ticket delivery via SMS/email.
QR Code Support	Replace or supplement barcode with QR codes for faster scanning at entry gates. Libraries: phpqrcode or endroid/qr-code.
Multi-Branch Support	Add a 'Branch' table allowing the system to manage multiple cinema locations under one admin account, with branch-level reporting.
Dynamic Seat Pricing	Implement tiered pricing: Front rows cheaper, premium center/back seats higher. Add VIP/Couple seat types with custom layouts.
Real-time Seat Map	Integrate WebSockets (Laravel Echo / Ratchet) to push seat status updates live — preventing stale seat maps when two staff book simultaneously.
Automated Showtime Status	Add a scheduled job (cron / Task Scheduler) to automatically set showtime Status='Ended' when End_Time passes, rather than manual updates.
Loyalty / Membership	Customer loyalty cards with points accumulation, discounts, and member-exclusive showtimes.
Mobile App	React Native or Flutter app for staff: scanning tickets via phone camera, checking seat availability, quick booking for walk-ins.
Advanced Reporting	Dashboard analytics: daily/weekly/monthly revenue charts (Chart.js), top movies by revenue, seat occupancy heat maps, export to Excel/PDF.
Inventory / Concessions	Add snack bar inventory module: products, stock levels, POS integration, and bundled movie+snack booking.
Automated Testing	Implement PHPUnit test suite covering AuthService, BarcodeService, and BookingController. Add Cypress E2E tests for critical booking flows.
Database Optimization	Add composite indexes: bookings(Status, Showtime), showtimes(Hall_ID, Start_Time), seats(Hall_ID, Status). Implement soft deletes properly.
Audit Logging	Create an audit_log table recording all INSERT/UPDATE/DELETE with user_id, timestamp, old_value, new_value for compliance and debugging.

Appendix A – Seed Data Reference

A.1 Default User Accounts

ID	Name	Username	Password	Role	Active
1	Walk-in Customer	(none)	(none)	Staff	Yes
2	Brent	(none)	(none)	Staff	NO
3	Administrator	admin	admin123	Admin	Yes
4	Staff User	staff	staff123	Staff	Yes

A.2 Cinema Halls

ID	Name	Total Seats	Is_Active	IsActive
1	Cinema 1	30	1	1
2	Cinema 2	30	1	1
3	Cinema 3	30	1	1

A.3 Movies

ID	Title	Genre	Rating	Duration	Price	Status
6	Spiderman	Action, Superpower	PG	60 min	₱250	Now Showing
7	365 DAYS	Romance, Steamy	R18	120 min	₱250	Ended
8	Sulasok	Thriller, Action	R18	60 min	₱250	Ended

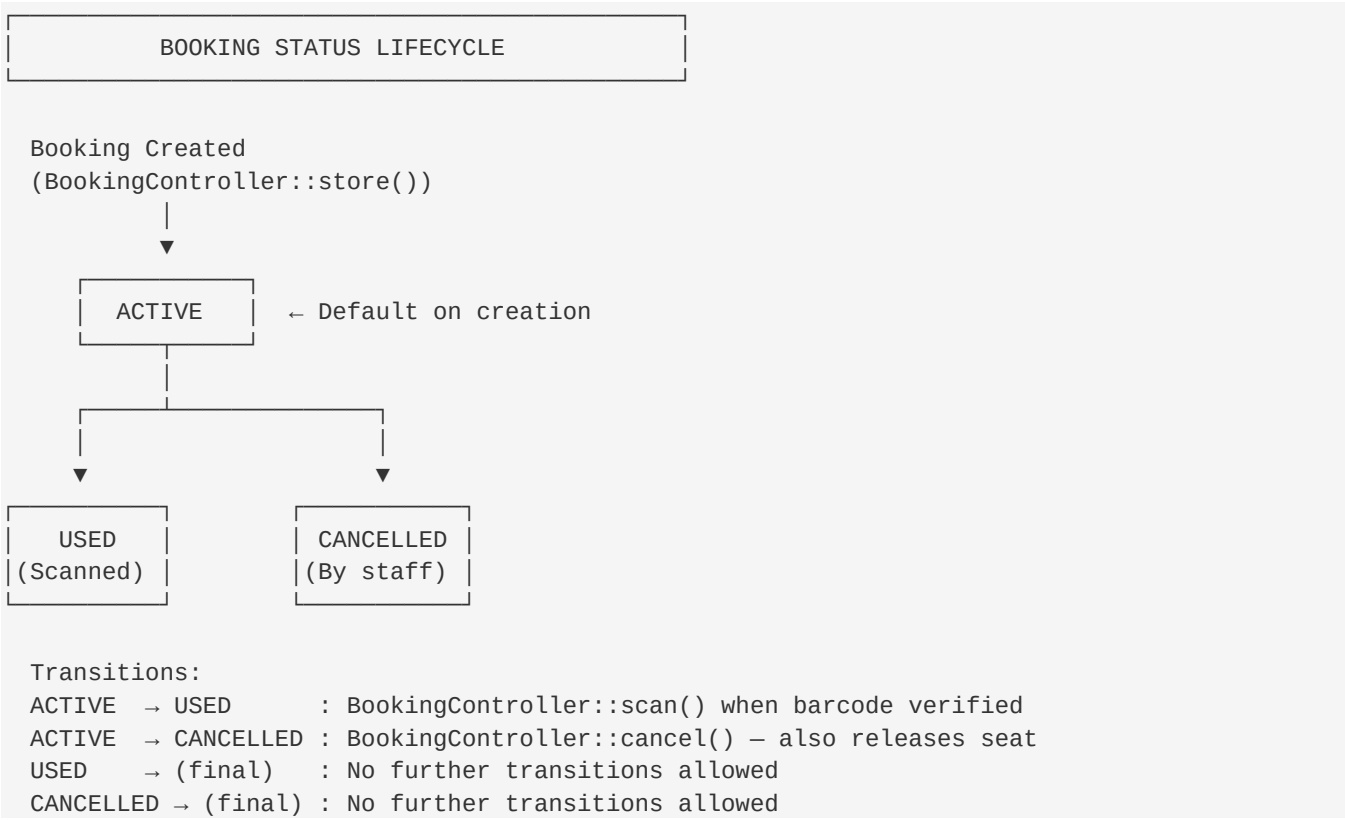
A.4 Seat Layout (Per Hall)

Each of the 3 halls has 30 seats arranged in 3 rows (A, B, C) × 10 columns (1–10):

```
Row A: A1  A2  A3  A4  A5  A6  A7  A8  A9  A10
Row B: B1  B2  B3  B4  B5  B6  B7  B8  B9  B10
Row C: C1  C2  C3  C4  C5  C6  C7  C8  C9  C10
```

Note: In Cinema 3, seat B7 is currently marked Reserved (Is_Occupied=1, Status='Reserved') from an active booking.

Appendix B – Booking Status Lifecycle



— End of Document —

CineScan Software Documentation · Version 1.0 · June 2026